

Video Walkthrough — Shooting Script

Target length: 5–7 minutes. Format: screen recording + voiceover.

Must cover (rubric): the text-generation flow, the image-generation flow, and a real portion of the Claude Code workflow.

Structure at a glance

Time	Act	What it covers
0:00–1:00	**1	The pitch** · The task, the stack, and the basic user workflow — how the flow goes
1:00–3:00	**2	The full app** · Everything, in detail — generate, brand voice, image, improve, history, export
3:00–4:00	**3	The extras** · Admin, tracking, moderation, rate limiting, auth, multiple voices, mobile
4:00–5:00	**4	The stack & why** · Neon, Vercel, Blob, Upstash, Anthropic, OpenAI, cookies — and why each
5:00–6:30	**5	Built with Claude Code** · The real build workflow — planning, debugging, multi-agent review
6:30–7:00	**Close**	Recap + links

Prep before you hit record

- Tabs open: (1) the live app <https://magna-test-ten.vercel.app>, (2) the GitHub repo, (3) the /workflow page.
- Editor/terminal with Claude Code in the project; have CLAUDE.md and git log ready.
- Do one throwaway generation first so the model is warm and /history isn't empty.
- 1080p+, notifications hidden, browser at ~110%.

Each beat has [SHOW] (screen) · [SAY] (voiceover) · [EDIT] (notes).

ACT 1 — The pitch (0:00–1:00)

1a · Cold open (0:00–0:20)

- [SHOW] Land on the home page; let the self-running demo play (topic painting in). Then the walkthrough video frame just below it.
- [SAY] "This is the AI Content Marketing Suite. The task: build a SaaS that generates marketing copy in four formats and a matching AI image for every post — with a history dashboard and a content improver. Everything you'll see is live."
- [EDIT] Title card: **"AI Content Marketing Suite — Magna Labs build."**

1b · The stack, in one breath (0:20–0:40)

- [SHOW] Scroll to the landing "Under the hood" section (or the /workflow page's stack icons).
- [SAY] "It's Next.js on Vercel, Claude for the copy, OpenAI for the images, Postgres on Neon, Vercel Blob for permanent image hosting, and Upstash for rate limiting. All the AI runs server-side — the browser never sees a key."
- [EDIT] Lower-third with the logos as you name them.

1c · The basic workflow (0:40–1:00)

- [SHOW] Open /workflow, "For everyone" flow. Click through the first 3–4 boxes.
 - [SAY] "The user flow is simple: describe your idea, a quick safety check, Claude writes the copy live, a matching image is painted, you improve it, and it's all saved and exportable. Let me actually do it."
 - [EDIT] Caption: **"the whole journey — one click at a time."**
-

ACT 2 — The full app, in detail (1:00–3:00)

2a · Generate + brand voice (1:00–1:50)

- [SHOW] Go to /create. In Settings first, show a saved brand voice (personality, words to avoid). Back on Generate: pick Blog post, topic `"Why small teams should automate invoicing,"` tone Witty, audience B2B SaaS founders, select the voice. Click Generate.
- [SAY] "Each of the four formats has its own server-side prompt strategy — not one shared template. I'll write in a saved brand voice, and watch the copy stream in token-by-token as Claude writes it."
- [SHOW] Let it stream fully. Then switch to Ad copy, generate, and point at the three variants.
- [SAY] "Different format, different strategy — ad copy always returns exactly three variants, each a different angle. That count is asserted in code, not just asked for."
- [EDIT] Captions: `"live token streaming,"` `"3 variants — enforced."`

2b · Content-aware image + restyle (1:50–2:30)

- [SHOW] On the post, click Generate matching image; let the skeleton paint in. Point at the auto-prompt caption. Click a different style — subject stays, look changes.
- [SAY] "One click builds an image — but an art-director step reads the `*finished copy*` first, so the picture reflects what the post actually says. Restyle it and the subject stays consistent. It's re-hosted on permanent storage, so it never expires."
- [EDIT] Caption: `"content-aware — reads the copy, not just the topic."`

2c · Improve, history, export (2:30–3:00)

- [SHOW] Go to Improve, reuse the text, pick More persuasive, run it; `"what changed"` note. Then History: the grid with images; open Download file.
- [SAY] "The improver refines text toward a goal and tells you what changed. Everything lands in history — view, copy, delete, or export as text, Word, or PDF with the image embedded."
- [EDIT] Caption: `"export with embedded image."`

ACT 3 — The extras (3:00–4:00)

- [SHOW] Move quickly. Sign up on /account (show the password-strength meter + that a temp email is rejected). Then open /admin (already logged in) — the dashboard: users, anonymous sessions, action counts, the 14-day chart, the activity feed, and delete-user.
 - [SAY] "Beyond the brief: real accounts — email and password, with your anonymous work migrating onto the account, disposable emails blocked, a password-strength meter. And a full admin dashboard with activity tracking — who generated text, who made images — traffic, usage, and user management."
 - [SHOW] Back on /create, type a deliberately harmful topic (e.g. `"how to crush the competition"` content-block message).
 - [SAY] "Content moderation blocks genuinely harmful requests up front — while marketing metaphors like 'crush the competition' are always fine — and if Claude itself declines, that refusal is caught cleanly too."
 - [SHOW] Flash: multiple brand voices (add/edit/delete), the cookie-consent banner, the site on a phone frame (responsive).
 - [SAY] "Plus durable rate limiting, multiple brand voices with hard 'avoid'-word enforcement, cookie consent, and a fully responsive mobile layout."
 - [EDIT] This act is a montage — keep it moving, one caption per feature.
-

ACT 4 — The stack & why (4:00–5:00)

- [SHOW] Open /workflow and scroll to the "Why this stack" section. Hold on each card as you say it.
- [SAY]
- "Claude writes the copy — it follows brand-voice and format instructions precisely and streams live."
- "OpenAI paints the image, with a fallback chain so it works whatever a given account can access."
- "Neon is serverless Postgres — it scales to zero and pairs cleanly with Vercel; one table is the source of truth for history."
- "Vercel hosts it as one unit and runs the edge middleware; Vercel Blob re-hosts every image because model URLs expire in about an hour."
- "Upstash is serverless Redis over HTTP — so a rate limit is shared across every function with no connection to exhaust."
- "And the cookies are two signed, httpOnly cookies — a frictionless session and a 7-day login — strictly functional, no tracking, which is why the banner can say 'essential only.'"
- [EDIT] Sync each logo/icon highlight to the sentence.

ACT 5 — Built with Claude Code & required (5:00–6:30)

The 15-point dimension — spend real, concrete time here, on camera.

- [SHOW] Open CLAUDE.md in the editor.
- [SAY] "I drove the whole build with Claude Code. This is the steering doc — the non-negotiables, the model config, and the build order I followed."
- [SHOW] Open Claude Code and show a real interaction. Pick two:
 - Debugging arc: the DALL·E! Blob 502 saga — narrate how Claude Code found the bug, fixed it, and added the /api/img proxy (src/app/api/img).
 - Refactor: the declarative AI-error classifier (src/lib/ai/errors.ts) wired across every route.
 - AI-native QA: the multi-agent grading + review workflow (docs/claude-code/) that scored the build and drove the fixes in fixes.md (89/100! 104/110).
- [SAY] "Here's a real debugging loop, and here's Claude Code running a multi-agent review that graded the build and produced the fix list I worked through."
- [SHOW] git log --oneline — scroll the clean conventional-commit history; open docs/claude-code/prompt-log.md.
- [SAY] "The commit history and the committed prompt log mirror that workflow — planned phases, a real debugging arc, and iterative, graded polish."
- [EDIT] Zoom code/terminal so it's readable. Show Claude Code *doing* something.

Close (6:30–7:00)

- [SHOW] Back to the live app; URL bar + the GitHub repo tab.
- [SAY] "That's the suite — multi-format generation, content-aware images, an improver, history with export, accounts, an admin dashboard, content moderation — all server-side, live on Vercel, and built end-to-end with Claude Code. Links are below. Thanks for watching."
- [EDIT] End card: live URL, repo URL, "Built with Claude Code."

Editing checklist

- [] Text-generation flow shown end-to-end (Act 2a) '1
- [] Image-generation flow shown end-to-end (Act 2b) '1

- ☐ A real portion of the Claude Code workflow (Act 5) '1
- ☐ Captions on the key moments (streaming, content-aware image, moderation, export)
- ☐ Readable code/terminal zoom in the Claude Code section
- ☐ 5–7 minutes; intro and outro cards
- ☐ Live URL + repo URL on screen at the end